# SDN Mininet Orange Problem

### Student Details

**Name:** Ramith M R

**SRN:** PES1UG24CS367

**Section:** 4-F

### 1. Problem Statement

The objective of this project is to implement a Software Defined Networking (SDN) solution using Mininet and a controller (POX).

The project demonstrates controller–switch interaction, flow rule installation, and monitoring of network behavior, specifically focusing on port status changes (up anddown events).

## 2. Objective

- To simulate a network using Mininet
- To implement a controller using POX
- To monitor port status changes dynamically
- To analyze network behavior during normal and failure conditions

# 3. Tools Used

- Mininet
- POX Controller
- OpenFlow Protocol
- Python 3.9 version
- Ubuntu Virtual Machine

# 4. Setup and Execution Steps

### Step 1: Start Controller

```
cd pox
python3.9 pox.py forwarding.l2_learning misc.port_monitor
```

### Step 2: Start Mininet

```
sudo mn --controller=remote --topo single,3
```

### Step 3: Verify Connectivity

```
pingall
```

### Step 4: Trigger Port Down Event

```
link h1 s1 down
```

### Step 5: Trigger Port Up Event

```
link h1 s1 up
```

### Step 6: View Flow Table

```
sudo ovs-ofctl dump-flows s1
```

# 5. Implementation Details

The controller monitors port status using OpenFlow events. When a port is modified, added, or deleted, the controller logs to the event. The  switch logic ensures packet forwarding by dynamically installing flow rules based on observed traffic.

# 6. Test Scenarios

## Scenario 1: Normal Operation

- All hosts are connected
- Packets are successfully transmitted
- No packet loss observed

**Command Used:**

```
pingall
```

*Normal connectivity (0% packet loss)*

```
*** Creating network
*** Adding controller
Unable to contact the remote controller at 127.0.0.1:6653
Connecting to remote controller at 127.0.0.1:6633
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
```

## Scenario 2: Port Failure

- Link between host and switch is disabled
- Communication fails
- Packet loss observed

**Command Used:**

```
link h1 s1 down
pingall
```

*Packet loss during failure*

```
mininet> link h1 s1 down
mininet> link h1 s1 up
mininet> link h1 s1 down
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X
h2 -> X h3
h3 -> X h2
*** Results: 66% dropped (2/6 received)
```

## Scenario 3: Recovery

- Link is restored
- Network resumes normal operation

**Command Used:**

```
link h1 s1 up
pingall
```

*Network recovery (0% packet loss)*

```
_learning misc.port_monitor
POX 0.7.0 (gar) / Copyright 2011-2020 James McCaule
WARNING:version:Support for Python 3 is experimenta
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 2] connected
INFO:misc.port_monitor:Port 1 modified
INFO:misc.port_monitor:Port 1 modified
INFO:misc.port_monitor:Port 1 modified
```

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> link h1 s1 down
mininet> link h1 s1 up
mininet>
```

# 7. Controller Output (Port Monitoring Logs)

*Controller showing port status changes*

```
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et
WARNING:version:Support for Python 3 is experimental.
INFO:core:POX 0.7.0 (gar) is up.
INFO:openflow.of_01:[00-00-00-00-00-01 1] connected
INFO:misc.port_monitor:Port 1 is UP
INFO:misc.port_monitor:Port 2 is DOWN
INFO:misc.port_monitor:Port 3 is DOWN
INFO:misc.port_monitor:Port 2 is UP
INFO:misc.port_monitor:Port 3 is UP
```

# 8. Flow Table Analysis

Command:

```
sudo ovs-ofctl dump-flows s1
```

*Flow table entries*

```
ramith-networks@ramith-networks-VirtualBox:~$ sudo ovs-ofctl dump-flows s1
 cookie=0x0, duration=4.930s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=0a:67:af:00:3a:2a,dl_dst=72:06:bd:82:23:c1,nw_src=10.0.0.1,nw_dst=10.0.0.2,nw_tos=0,icmp_type=8,icmp
 cookie=0x0, duration=4.928s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=72:06:bd:82:23:c1,dl_dst=0a:67:af:00:3a:2a,nw_src=10.0.0.2,nw_dst=10.0.0.1,nw_tos=0,icmp_type=0,icmp
 cookie=0x0, duration=4.921s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=0a:67:af:00:3a:2a,dl_dst=a6:30:bb:0f:31:58,nw_src=10.0.0.1,nw_dst=10.0.0.3,nw_tos=0,icmp_type=8,icmp
 cookie=0x0, duration=4.919s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=a6:30:bb:0f:31:58,dl_dst=0a:67:af:00:3a:2a,nw_src=10.0.0.3,nw_dst=10.0.0.1,nw_tos=0,icmp_type=0,icmp
 cookie=0x0, duration=4.910s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=72:06:bd:82:23:c1,dl_dst=0a:67:af:00:3a:2a,nw_src=10.0.0.2,nw_dst=10.0.0.1,nw_tos=0,icmp_type=8,icmp
 cookie=0x0, duration=4.909s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=0a:67:af:00:3a:2a,dl_dst=72:06:bd:82:23:c1,nw_src=10.0.0.1,nw_dst=10.0.0.2,nw_tos=0,icmp_type=0,icmp
 cookie=0x0, duration=4.902s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=72:06:bd:82:23:c1,dl_dst=a6:30:bb:0f:31:58,nw_src=10.0.0.2,nw_dst=10.0.0.3,nw_tos=0,icmp_type=8,icmp
 cookie=0x0, duration=4.901s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=a6:30:bb:0f:31:58,dl_dst=72:06:bd:82:23:c1,nw_src=10.0.0.3,nw_dst=10.0.0.2,nw_tos=0,icmp_type=0,icmp
 cookie=0x0, duration=4.892s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=a6:30:bb:0f:31:58,dl_dst=0a:67:af:00:3a:2a,nw_src=10.0.0.3,nw_dst=10.0.0.1,nw_tos=0,icmp_type=8,icmp
 cookie=0x0, duration=4.891s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=0a:67:af:00:3a:2a,dl_dst=a6:30:bb:0f:31:58,nw_src=10.0.0.1,nw_dst=10.0.0.3,nw_tos=0,icmp_type=0,icmp
 cookie=0x0, duration=4.884s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=a6:30:bb:0f:31:58,dl_dst=72:06:bd:82:23:c1,nw_src=10.0.0.3,nw_dst=10.0.0.2,nw_tos=0,icmp_type=8,icmp
 cookie=0x0, duration=4.882s, table=0, n_packets=1, n_bytes=98, idle_timeout=10, hard_timeout=30, priority=6
,dl_src=72:06:bd:82:23:c1,dl_dst=a6:30:bb:0f:31:58,nw_src=10.0.0.2,nw_dst=10.0.0.3,nw_tos=0,icmp_type=0,icmp
ramith-networks@ramith-networks-VirtualBox:~$
```

### Explanation:

- Flow entries are installed dynamically by the controller
- Each rule contains:
  - Match field (e.g., input port)
  - Action (e.g., forward to another port)
- This demonstrates how SDN separates control and data plane

# 9. Observations

- The controller detects port status changes in real-time
- When a port goes down, communication is interrupted
- Flow rules are dynamically updated based on traffic
- Network recovers immediately after port restoration

# 10. Conclusion

This project demonstrates SDN concepts using Mininet and POX.

It shows us how a controller can monitor and control network behavior dynamically.

The implementation highlights the flexibility and programmability of SDN in handling network events like port failures.